Politecnico di Milano

Ingegneria del Software – a.a. 2004/05

Appello del 9 settembre 2005

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 28/30.

Esercizio 1: .....

Esercizio 2: .....

Totale .....

Esercizio 1 (punti 20)

Si vuole realizzare un insieme di elementi "pesati", in cui ogni oggetto dell'insieme è costituito da un nome e da un peso, che può andare da 0 a 63. Le operazioni che si possono effettuare su un insieme sono le seguenti: aggiungere un nuovo elemento (con il relativo peso); togliere uno o più elementi tra quelli di peso maggiore presenti nell'insieme; cambiare il peso di uno degli elementi presenti nell'insieme.

Per realizzare gli elementi da inserire nell'insieme si sceglie di usare il seguente tipo **Elemento**.

```
public class Elemento {
  //@public invariant nome != null && 0 <= peso && peso < 64;
  public final String nome;
  public final int peso;

  //@ensures n != null && 0 <= peso && peso < 64 &&
  //@        nome == n && peso == p;
  //@signals (IllegalArgumentException e)
  //@        n == null || peso < 0 ||  peso >= 64;
  public Elemento(String n, int p){ /* ... */ }

  public boolean equals(Elemento e){
    return e.nome.equals(nome) && e.peso == peso;
  }
  public boolean equals(Object e){
    if(e instanceof Elemento) return equal((Elemento)e);
    return false;
  }
}
```

Sia data inoltre la seguente specifica di un tipo **StringSet**.

```
public class StringSet {
  // OVERVIEW: insiemi di stringhe illimitati e modificabili;
  // per es.: {alfa, pippo, pallino}

  //@ensures (*\result è true sse x è fra gli elementi di this*);
  public /*@ pure @*/ boolean isIn (String x){ /* ... */ }

  //@ensures (*\result è cardinalità di this *);
  public /*@ pure @*/ int size(){ /* ... */ }

  //@ensures (\forall String y;;!this.isIn(y));
  public StringSet(){ /* inizializza this come insieme vuoto */ }

  //@ensures this.isIn(x) &&
  //@         (\forall String y; x!=y;
  //@                   \old(this.isIn(y)) <==> this.isIn(y));
  public void insert(String x){ /* inserisce x in this */ }

  //@ensures !this.isIn(x) &&
  //@        (\forall String y; x!=y; \old(this.isIn(y)) <==> this.isIn(y));
  public void remove(String x){ /* rimuove x da this */}

}
```

1. Si completi la seguente dichiarazione della classe **WeightedSet** con le opportune specifiche JML.

```
public class WeightedSet {
  // OVERVIEW: insiemi di elementi pesati <nome, peso> tali per cui ad ogni "nome"
  //           nell'insieme e' associato al massimo un peso (non sono presenti
  //           cioe' nell'insieme due coppie con nome uguale e peso diverso).
  //@public invariant
  //@(\forall  Elemento x; isIn(x);
  //@  (\forall Elemento y; isIn(y) && !x.equals(y);
  //@     x.nome != y.nome))


  //@ensures (*\result è true sse x è fra gli elementi di this*);
  public /*@ pure @*/ boolean isIn (Elemento x){ /* ... */ }

  //inizializza this come insieme vuoto
  //@ensures
  //@  (\forall Elemento y;; !this.isIn(y))
  public WeightedSet(){ /* ... */ }

  //Aggiunge x a this. x deve essere diverso da null e tale che non ci sia in
  //this un elemento con lo stesso nome di x, anche se con peso diverso.
  //@requires
  //@ x!=null && !(\exists Elemento y; isIn(y); y.nome==x.nome && y.peso!=x.peso)

  //@ensures
  //@isIn(x) && (\forall Elemento y;; isIn(y) <==> y.equals(x) || \old(isIn(y)))
  public void add(Elemento x){ /* ... */ }

  //Ritorna un qualunque sottoinsieme di nomi presi tra quelli di peso maggiore
  //presenti in this. Gli elementi i cui nomi vengono ritornati sono eliminati
  //da this.
  //@ensures
  //@(\forall String s; \result.isIn(s);
  //@   (\exists Elemento x;;
  //@              \old(isIn(x)) &&
  //@              (\forall Elemento y; \old(isIn(y)); (y.peso <= x.peso))) &&
  //@   !(\exists Elemento x; isIn(x); x.nome==s))
  public StringSet get(){ /* ... */}

  //Cambia il peso associato all'elemento di nome n in p; siccome gli attributi
  //della classe Elemento sono final, si noti che per "modificare" un peso
  //occorre in realta' creare un nuovo oggetto Elemento con il nuovo peso.
  //Se nell'insieme non e' presente alcun elemento con nome n, oppure se il peso
  //p non e' un peso valido, solleva un'eccezione checked WeightedSetExc.
  //@requires  n!=null
  //@ensures 0<=p && p<64 &&
  //@        (\exists Elemento x; \old(isIn(x)); x.nome==n) &&
  //@        (\exists Elemento x;;
  //@            isIn(x) && x.nome==n && x.peso==p &&
  //@            (\forall Elemento y; !y.equals(x) && isIn(y); y.nome!=x.nome)) &&
  //@        (\forall Elemento y; y.nome!=n; \old(isIn(y)) <==> isIn(y))
  //@signals (WeightedSetExc e): p<0 || p>=64 ||
  //@                             !(\exists x:Elemento; \old(isIn(x)); x.nome=n)
  public void cambiaPeso(String n, int p) throws WeightedSetExc { /* ... */}
}
```

2. Come cambiano (se cambiano) la dichiarazione e la specifica JML del metodo **add** se si adotta uno stile di programmazione difensiva, cioè si sceglie di sollevare un'eccezione **WeightedSetExc** nel caso in cui l'oggetto x di tipo **Elemento** abbia un nome che è già presente nell'insieme pesato, ma associato ad un peso diverso da quello di x? Se ne riscrivano dichiarazione e specifica in modo da rispecchiare la modifica.

```
  //@ensures: x!=null &&
  //@     !(\exists Elemento y; \old(isIn(y)); y.nome==x.nome && y.peso!=x.peso) &&
  //@     isIn(x) &&
  //@     (\forall Elemento y; ;isIn(y)<==> y.equals(x)|| \old(isIn(y)))
  //@signals (WeightedSetExc e):
  //@    x==null ||
  //@    (\exists Elemento y; \old(isIn(y)); y.nome==x.nome && y.peso!=x.peso)
```

3. Si decide di implementare l'insieme mediante un array di 64 insiemi di tipo **StringSet**. Ogni insieme corrisponde ad un peso tra 0 e 63, e contiene tutti (e soli) i nomi degli elementi di quel peso presenti nell'insieme. Si completi la dichiarazione sottostante con l'indicazione del rep invariant della classe **WeightedSet**.

```
public class WeightedSet {
  //rep:
  private StringSet elPesati[64];
  //@private invariant
  //@(\forall int i; 0<=i && i<64; elPesati[i]!=null) &&
  //@(\forall int i; 0<=i && i<64;
  //@    (\forall int j; i<j && j<64;
  //@    !(\exists String x; ; elPesati[i].isIn(x) && elPesati[j].isIn(x))))
}
```

4. Si implementi il metodo **add** della classe **WeightedSet** (*una* versione a scelta tra quella del punto 1 e quella del punto 2).

Implementiamo la versione del punto 2, quella che lancia l'eccezione.

```
void add(Elemento x) throws WeightedSetExc {
  if(x==null) throw new WeightedSetExc();
  for(int i=0; i<64; i++){
    if((i!=x.peso && elPesati[i].isIn(x.nome))
      throw new WeightedSetExc();
  }

  elPesati[i].add(x.nome);
}
```

5. Si dimostri che l'implementazione di **add** del punto 4 mantiene l'invariante di rappresentazione scritto al punto 3, e che rispetta la specifica del metodo.

Il rep invariant è garantito dal ciclo for. Infatti se il rep invariant è vero all'ingresso del metodo ciò significa che non ci sono in *elPesati* array diversi contenenti lo stesso nome. Ora, x.nome deve andare nel set di indice x.peso, per cui, perchè il rep invariant sia garantito, x.nome non comparire in nessuno degli array con indice diverso da n.peso. Ciò è garantito dal ciclo for quindi il rep invariant è mantenuto.
D'altro canto, la specifica è garantita in quanto:

- la clausola della signals corrisponde esattamente al primo if e alla condizione per cui, nel primo ciclo for, viene sollevata un'eccezione;
- le prime due condizioni della ensures sono vere all'uscita del ciclo for;
- la terza condizione è garantita dalla add;
- la quarta condizione è garantita dal fatto che non viene rimosso nulla dall'insieme.

6. Sia data una classe **WeightedSetAll** che estende **WeightedSet** e ridefinisce il metodo **get** in modo che esso ritorni *tutti* gli elementi di this che hanno il peso più grande tra quelli presenti nell'insieme (se per esempio il peso più grande presente nell'insieme è 50, il metodo ritorna tutti gli elementi dell'insieme di peso 50).
   a. Si dia la specifica JML del nuovo metodo **get**.
   b. La classe **WeightedSetAll** rispetta il principio di sostituibilità? Si motivi brevemente, ma con precisione, la risposta.

Basta aggiungere alla clausola **ensures** del padre

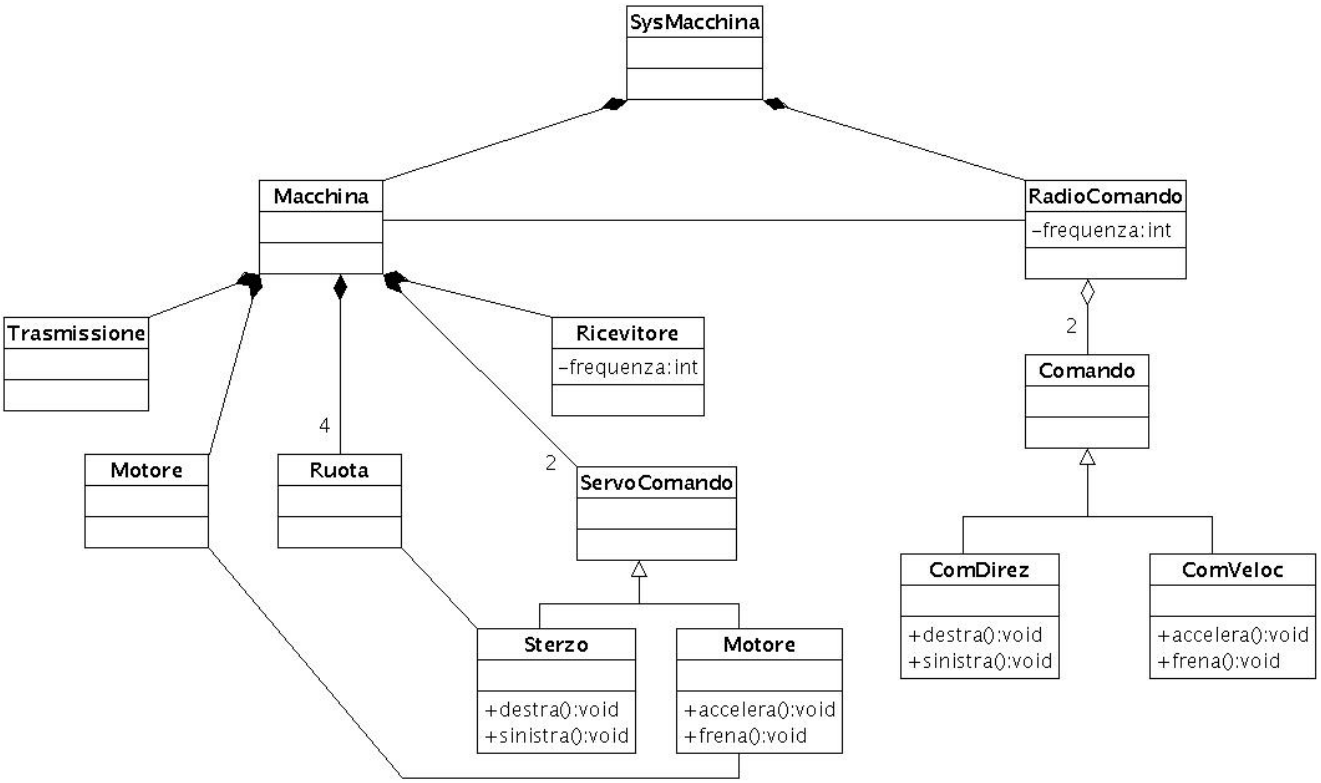
```
//@ensures: ... /* la clausola ensures del padre */ &&
//@  (\forall Elemento x; \old(isIn(x)) &&
//@                       (\forall Elemento y; \old(isIn(y)); y.peso<=x.peso);
//@           \result.isIn(x.nome));
```

rispetta il principio di sostituibilità perché sono verificate sia la regola della segnatura, che quella dei metodi, che quella delle proprietà. In particolar modo, la regola della segnatura e quella delle proprietà sono banalmente verificate. Quella dei metodi è verificata in quanto le postcondizioni di **WeightedSetAll** sono più forti di quelle di **WeightedSet** (infatti **WeightedSetAll** precisa quali elementi vengono ritornati, mentre il padre questo lo lasciava indeterminato). Ciò è evidenziato dal fatto che la specifica del nuovo metodo è data dalla vecchia specifica, con una *aggiunta* di condizioni.

Esercizio 2 (punti 8)

Una macchina radiocomandata è formata da due parti: la macchina stessa ed il radiocomando. Ci sono 2 tipi di comandi, quelli di direzione e quelli di velocità. Mediante i comandi di direzione si può scegliere se girare a destra o a sinistra; i comandi di velocità invece permettono di scegliere se accelerare (in questo caso, di quanto accelerare) o se frenare. Ogni radiocomando può essere configurato per funzionare su frequenze diverse.
La macchina, invece, si compone di un motore, di un sistema di trasmissione, di 4 ruote, di un ricevitore/controllore, e di 2 servocomandi. I servocomandi possono essere di 2 tipi, di direzione o di velocità. Una macchina ha un servocomando per la direzione (piazzato sulle ruote anteriori), e uno per la velocità (installato sul motore). Il ricevitore/controllore della macchina può anch'esso essere configurato per funzionare su diverse frequenze (la frequenza del ricevitore deve essere la stessa del radiocomando perchè i due possano comunicare).

a. Si rappresenti mediante un diagramma delle classi UML il sistema di cui sopra.



b. Si scriva un Sequence Diagram UML che descrive la seguente situazione: il guidatore accende la macchina, quindi inizialmente accelera in direzione rettilinea, quindi curva a destra; durante la curva frena, quindi rimette le ruote in posizione rettilinea. Dopo avere rimesso le ruote in posizione rettilinea frena, quindi spegne il motore.

